

Guía de Flujo de Trabajo y Gestión de Ramas

Esta guía documenta la estructura de ramas del proyecto y establece el flujo de trabajo estándar que todos los desarrolladores deben seguir para garantizar una integración continua, ordenada y libre de conflictos.

[!NOTE] Actualmente el proyecto cuenta con las ramas base: `main`, `DEV`, `QA` y ramas de iteración como `SPRINT-1`.

1. Estructura de Ramas

El repositorio está organizado en las siguientes ramas principales:

- `main`: Refleja el estado en **Producción**. El código aquí es completamente estable y funcional. No se trabaja directamente sobre esta rama.
- `QA` (Quality Assurance): Entorno de **Pruebas**. Se utiliza para que el equipo de calidad valide los incrementos antes de pasarlos a producción.
- `DEV` (Development): Entorno de **Desarrollo/Integración**. Contiene las últimas características y correcciones aprobadas que formarán la próxima versión.
- `SPRINT-{N}` (Ej. `SPRINT-1`): Rama de la **iteración actual**. Agrupa todo el trabajo que se está realizando en el sprint activo. Nace de `DEV`.

2. Flujo de Trabajo del Desarrollador (Historias de Usuario)

Cada vez que se asigne una Historia de Usuario (HU) o tarea, el desarrollador debe seguir este proceso exacto:

Paso 1: Preparar el entorno local

Antes de iniciar, asegúrate de estar en la rama del sprint actual y tener los últimos cambios del repositorio remoto:

```
git checkout SPRINT-1
git pull origin SPRINT-1
```

Paso 2: Crear la rama de la Historia de Usuario (HU)

Nunca trabajes directamente sobre la rama `SPRINT-1`. Crea una rama nueva específica para tu tarea a partir del sprint activo. Usa una nomenclatura clara, como `feature/HU-{ID}-{breve-descripcion}`:

```
git checkout -b feature/HU-101-crear-login
```

Paso 3: Desarrollar y hacer Commits

Realiza tu trabajo y guarda los cambios con mensajes de commit claros y descriptivos que hagan referencia a la HU:

```
git add .
git commit -m "feat(HU-101): agrega formulario de autenticación"
```

Paso 4: Subir los cambios y crear un Pull Request (PR)

Una vez finalizado el trabajo y probada tu HU localmente, sube tu rama al repositorio remoto:

```
git push origin feature/HU-101-crear-login
```

[!IMPORTANT] Ve a la plataforma del repositorio (GitHub/GitLab/Bitbucket) y abre un **Pull Request (PR)** desde tu rama `feature/HU-101-crear-login` **hacia la rama** `SPRINT-1` .
Solicita la revisión de al menos un compañero de equipo antes de integrar.

3. Ciclo de Vida de Integración (Promoción del Código)

Así es como fluye el código desde tu computadora hasta producción:

```
gitGraph
    commit id: "Initial"
    branch DEV
    commit id: "DEV Base"
    branch QA
    commit id: "QA Base"
    checkout main
    branch SPRINT-1
    checkout SPRINT-1
    commit id: "Inicio Sprint 1"
    branch HU-101
    branch HU-102
    checkout HU-101
    commit id: "Trabajo HU-101"
    checkout SPRINT-1
    merge HU-101
    checkout HU-102
    commit id: "Trabajo HU-102"
    checkout SPRINT-1
    merge HU-102
    checkout DEV
    merge SPRINT-1
    checkout QA
    merge DEV
```

```
checkout main  
merge QA
```

1. **HU-XXX → SPRINT-1** : Cada desarrollador integra su HU al sprint actual tras ser aprobada por Code Review.
2. **SPRINT-1 → DEV** : Al finalizar el sprint, la rama del sprint se fusiona con `DEV` para su integración general.
3. **DEV → QA** : El código de `DEV` pasa a `QA` para iniciar las pruebas exhaustivas (Regresión, UAT, etc.).
4. **QA → main** : Una vez que QA aprueba la versión, se fusiona a `main` y se realiza el despliegue a producción.

4. Buenas Prácticas y Reglas Clave

[!WARNING]

- **Nunca hacer `push` o `commit` directo a `main`, `QA`, `DEV` o `SPRINT-X`.** Todo debe ser mediante Pull Requests.
- **Mantén tu rama actualizada.** Si tus compañeros han fusionado cambios en `SPRINT-1`, trae esos cambios a tu rama para evitar conflictos: `git fetch && git rebase origin/SPRINT-1` (o `git merge origin/SPRINT-1`).
- **Elimina ramas obsoletas:** Una vez que tu PR a `SPRINT-1` ha sido mergeado, elimina tu rama local y remota `feature/HU-XXX` para mantener el repositorio limpio.